DAT321/DIT843: Quality Assurance and Testing

Questions about the exam contact:
Examiner: Francisco Gomes, tel. 073 856 4439, francisco.gomes@cse.gu.se

- Francisco or Sabina will visit the exam halls around 09:30 and 11:00.

No aids are allowed for this exam.

The questions in this exam refer to the **ISO 25010:2011** that categorises four types of quality: process, internal, external and quality in use. The exam also covers the eight attributes for product quality.

Important advice:

- Each question has points assigned shown in the square brackets. When the question is broken down into smaller sub-questions the part of the points for those specific sub-questions are also shown. Example: 1. [10 pts]. 1a. [2 pts] 1b. [8 pts].
- You must write correct, clear, readable, understandable, and unambiguous answers. If we cannot easily read what you are trying to write, then we cannot positively judge your knowledge or your learning of the topic.
- Motivate your answers (a simple statement of facts not answering the question is invalid). Whenever possible refer to modules, features and information about the software product or software development process related to the questions. Avoid circular definitions and vague / generic answers.
- Start each question on a new paper. Sort your answers in order (by question and sub-task) before handing them in. Write only on one side of the paper.
- Write your student code on each page and put the number of the question on every paper. The exam is anonymous, so do not leave any information that can reveal your identity.

The points and the corresponding grade are presented below (100 points in total):

- 00 – 49: U (Fail)
- 50 - 69: 3 (Pass)
- 70 - 84: 4 (Pass with credit)
- 85 - 100: 5 (Pass with distinction)

The exam review is scheduled for:
Date: 2025-05-05 between 11:00 – 12:00.
Location: Jupiter room 400 (Jupiter building 4th floor – Software Engineering)

Question 1. [Total: 25 pts] Using the context below, answer the following questions:

You work for **FitTrack Innovations**, a company developing an advanced **fitness and health monitoring platform**. The platform includes a mobile app and a wearable device (like a smartwatch). Users can track workouts, monitor heart rate, get sleep analysis, and receive personalized health advice. The platform supports syncing with external apps such as Google Fit and Apple Health, and updates itself regularly with new fitness programs and improvements. The users of the platform are everyday individuals seeking to monitor and improve their health, ranging from beginners to professional athletes.

Some features and characteristics of the platform are mentioned below:

- **F1**: Users must create an account with multi-factor authentication (MFA) to access personal health data.
- **F2**: The app interface provides a dashboard showing real-time heart rate, calories burned, and steps taken.
- **F3**: The wearable device automatically syncs with the mobile app via Bluetooth or Wi-Fi.
- **F4**: Sleep tracking results are presented with easy-to-understand charts and health insights.
- **F5**: The platform sends personalized notifications (e.g., hydration reminders, exercise tips) and users can customize which notifications they want to receive.
- **F6**: Users can export their fitness data in standardized formats (e.g., CSV, JSON) to share with healthcare providers.
- **F7**: A new software update with bug fixes and new fitness programs is released every two months.
- **F8**: The system supports integration with third-party fitness apps and devices like Garmin, Fitbit, Google Fit, and Apple Health.
- **F9**: The app is designed to work on Android, iOS, and selected smartwatches, with interfaces adapted for different screen sizes.
- **F10**: The wearable must provide accurate tracking of physical activity even in offline mode (no connection to app).

Note: For the questions below, you are allowed to add assumptions about the system (e.g., expected user load, device specs, etc.). However, any assumption must (i) be clearly stated and (ii) not contradict any points above.

a. **[5 pts]** Map each feature listed above, to one or more quality attribute(s) from ISO25010:2011 that can be used to measure its quality. For this question, you can ignore functional suitability and use the remaining 7 quality attributes.

Note: Your answer can simply link between the quality attribute and the feature. There is no need to justify your mapping.

0.5 points for each correct identification, partical credit for one correct and another unrelated (rounded down).

Feature	Expected Quality Attribute(s)
F1 (MFA account creation)	Security
F2 (dashboard real-time data)	Performance Efficiency, Usability
F3 (wearable device sync)	Compatibility, Reliability
F4 (sleep tracking charts)	Usability
F5 (personalized notifications)	Usability, Maintainability
F6 (export fitness data)	Portability, Compatibility
F7 (regular updates)	Maintainability, Reliability
F8 (third-party integration)	Compatibility
F9 (multi-device GUI)	Portability, Usability
F10 (offline tracking)	Reliability

b. **[8 pts]** Select **two** quality attributes from the list above. For each selected attribute, describe a **quality scenario** by clearly specifying: (i) Source, (ii) Stimulus, (iii) Artefact, (iv) Environment, (v) Response, and (vi) Response Measure.

+4 pts for each scenario. The scenario will be evaluated based on how testable it is, and whether its elements map to the chosen attribute. Below, we included two examples of quality scenarios.

Portability (could also have been usability)

- Source: User who switches between devices
- Stimulus: User installs the FitTrack app on a new Android tablet
- Artefact: FitTrack mobile application
- Environment: New Android device, different screen size and OS version
- Response: The app installs successfully, adapts the interface to the tablet, and retains all synced user data
- Response Measure: 100% successful installation. Layout correctly adjusted in under 5 seconds

Performance Efficiency:

- Source: User checking workout summary after a run
- Stimulus: User opens the real-time dashboard in the app to view heart rate, calories, and steps immediately after finishing a workout
- Artefact: FitTrack dashboard component
- Environment: Mobile device with moderate internet connection (4G)
- Response: The dashboard loads and displays updated workout metrics quickly
- Response Measure: Dashboard metrics load and refresh in under 2 seconds.

c. **[12 pts]** Your development team for **FitTrack** is concerned about **technical debt** increasing as new features are added every month. During a team meeting, two suggestions are proposed:

- **John** suggests that the team should **prioritize refactoring parts of the wearable device codebase** that are difficult to maintain.
- **Mary** suggests that the team should **improve bug monitoring and fix minor bugs** quickly before they accumulate and cause bigger issues later.

Both John and Mary are trying to reduce risks for the project. Answer: (i) Which suggestion would be **more effective** for **managing technical debt**? (ii) Which one could also help **prevent technical debt from growing** if handled properly?

Clearly explain your reasoning by linking your answer to the concepts of technical debt, internal quality, and external quality.

(i) 6 pts: +1 choose John's suggestion. +2 Tehc debt definition. +3 internal quality.
Example answer: **John's suggestion would be more effective** for managing technical debt. Refactoring parts of the wearable device codebase directly **improves the internal quality** of the system. Technical debt mainly affects internal quality — it is about **how easy it is for developers to evolve the system safely** and efficiently.
Refactoring improves code readability, structure, and maintainability, directly addressing this internal aspect.

(ii) **6 pts: +3 Argue for Mary in complement to Johns. +3 Connection to External Quality.**

Mary's suggestion could help **prevent** technical debt from growing if handled properly. Improving bug monitoring and fixing minor bugs early can prevent the system from accumulating lots of small issues that might later turn into larger architectural problems. Although fixing **minor bugs focuses more on external quality** (what users experience), maintaining a consistent habit of addressing small problems early can also support good internal quality over time. However, while Mary's approach is **good for stability and quality assurance**, it is not directly attacking existing technical debt. It is more about risk prevention and external quality management (keeping the system working well for users),

---

Question 2 [50 points]: Based on your knowledge of Property-based testing (PBT), Black-box, and White-box Testing, answer the questions below. We include the Software Under Test (SUT), Java code, some examples of test cases, and also some examples of methods below that can be used when writing your properties.

Function	Description
`length(String[]): int`	Returns the number of elements in the array.
`sumLength(String[]): int`	Returns the summed length of all strings in the array.
`length(String): int`	Returns the number of characters in the string.
`contains(String, String): boolean`	Returns true if the second string is contained in the first.
`toLowerCase(String): String`	Converts a string to lowercase.

Specification: The function `filterAndJoinStrings(String[] words, int minLength)` receives an array of strings and an integer `minLength`. It returns a new string created by joining all strings in the array that have a length **greater than or equal** to minLength, separated by commas **“,”**.

- The array only contains valid strings (i.e., non-empty and non-null strings).
- Only lowercase alphabetic characters (a–z) appear in the input strings.
- No numbers, punctuation, or spaces are in the input strings.
- If the input array is `null`, the function must throw an exception with the message: "Invalid array of strings.".
- If no strings satisfy the condition, the function must return the empty string `""` .
- Any null elements inside the array must be ignored.

```java
public static String filterAndJoinStrings(String[] words, int minLength){
    if (words == null) {
        throw new IllegalArgumentException("Invalid array of strings.");
    }
    String result = "";
    boolean first = true;

    // In Java, "+" concatenates two strings
    for (String str : words) {
        if (str.length() >= minLength) {
            if (!first) {
                result = result + ",";
            }
            result = result + str;
            first = false;
        }
    }
    return result;
}
```

TCs	Input	Expected Output
T1	`["apple", "bat", "cat", "dog"], 4`	`"apple"`
T2	`["one", "two", "three"], 3`	`"one,two,three"`
T3	`["sun", "moon", "star"], 5`	`""`
T4	`[], 2`	`""`
T5	`["hello", null, "world"], 4`	`"hello,world"`
T6	`["a", "ab", "abc", "abcd"], 3`	`"abc,abcd"`
T7	`["oneword"], 7`	`"oneword"`
T8	`["tiny", "big"], 10`	`""`
T9	`null, 3`	`Exception: "Invalid array of strings."`
T10	`["a", "b", "c"], 0`	`"a,b,c"`
T11	`["x", "y", "z"], 1`	`"x,y,z"`

a. **[15 pts]** Create exactly five (5) properties for `filterAndJoinStrings(String[] words, int minLength)`. Feel free to use any of the support functions for arrays of integers listed in the beginning of this question. You can write the properties in natural language, mathematical notation, or pseudocode.

Note: Your properties will be evaluated based on their variety, clarity, and coverage.

+3 per property. Remember to include both input parameters in the property, and to avoid being redundant to the specificaion. You could also use the sumLength, function and the minlength to calculate the size of the resulting string in a few cases.

ID	Property
P1	If the input array is null, **and any minLenght**, the function throws IllegalArgumentException.
P2	If the input array is empty ([]), **and any minLenght**, the function returns the empty string "".
P3	If minLength <= 1, and **any non-empty words**, the result includes all strings in the input array, joined by commas, in original order.
P4	**For any non-empty word strign, and minlength > 1**, the output string must not start or end with a comma, and must not contain two consecutive commas ,,.
P5	If no string satisfies the minLength condition > 1, the function returns the empty string "".
P6	The order of strings in the output must match the order of qualifying strings in the input array.

b. [12 pts] Create four equivalent partition classes for the function filterAndJoinStrings.

+3 points per partition. The partitions should define the set of input values, as well as the expected behavior of the SUT. The partitions should not be very similar to the specification, so focus on describing constrains on either of the parameter to split the input space.

Some examples of partitions:

Class	Description
P1	Input array is null → should throw IllegalArgumentException.
P2	Input array is empty ([]) → should return the empty string "".
P3	All strings are shorter than minLength → should return the empty string "".
P4	All strings satisfy the minLength condition → all strings are joined in the original order, separated by commas.
P5	minLength <= 1 → all non-null strings are joined regardless of their length.
P6	String array with just one element → there is no comma in the output

c. [13 pts] Using the partition classes that you defined above, create test cases that reveal four boundaries. You must indicate the corresponding equivalent partition classes of each boundary listed.

+3 per boundary. +1 in the variety of boundaries and partitions coverage. Some examples of boundaries below.

- Boundary 1: Boundary between null input and empty array
  - **P1**: filter(null, 3) → Exception: "Invalid array of strings."
  - **P2**: filter([], 3) → ""
- Boundary 2: Boundary between empty array and array with one string
  - **P2**: filter([], 2) → ""
  - **P6**: filter(["a"], 5) → ""
- Boundary 3: Boundary between all strings too short and one string satisfying minLength
  - **P3**: filter(["a", "b"], 3) → ""
  - **P4**: filter(["a", "bat"], 3) → "bat"
- Boundary 4: Boundary between all strings satisfying minLength and one too short
  - **P4**: filter(["tree", "bush"], 3) → "tree,bush"
  - **P5**: filter(["tree", "hi"], 1) → "tree"
- Boundary 5: Boundary between minLength = 1 and no string meets minLength
  - **P5**: filter(["a", "b", "c"], 1) → "a,b,c"
  - **P3**: filter(["a", "b", "c"], 2) → ""

d. **[10 pts]** For the code above, (i) create two mutants using different mutation operators. You only need to write the specific line of code that you mutated. (ii) Specify which mutation operators you used, and (iii) for each mutant, create one test case that kills it (feel free to use the ones provided in the test suite above). For the test cases, you must specify **both** the expected output and the actual output produced by the mutant.

+5 each: +1 Mutant, +1 Operator, +3 Test case (Input, Expected, Actual)

```
Mutant 1:
+1 (i) Line 10. Change to if (str.length() > minLength)
+1 (ii) Relational operation replacement (ROR)
+3 (iii) TC: filter(["cat"], 3). Expected: "cat", Actual: ""

Mutant 2:
(i) Change Line 11 to if (first) { (remove the ! )
(ii) Conditional Operator Replacement (COR)
(iii) TC: filter( ["hi", "hello"], 2). Expected: "hi,hello". Actual:
",hihello"
```

Question 3 [25 pts]: Using your knowledge of test suite minimization and test prioritization for regression testing, answer the following:

The table below shows the traceability between each test case (row) and the code elements (methods) (columns) they cover. For example, T1 covers Methods M1 and M4.

Test Case ID	M1	M2	M3	M4	M5
T1	x			x	
T2	x	x			
T3			x		
T4		x	x	x	
T5	x				
T6			x		x
T7		x		x	

a. **[10 pts]** Create two test suites: (i) Use test suite minimsation to create a test suite that covers all code elements (M1, M2, M3, M4, M5) at least once. (ii) Create a prioritized test suite that covers as many different methods. For both test suites, your answer must list the selected tests, and justify your choice of tests.

+5 per test suite. +1 choice of tests, +1 definition, +3 justification.

Minimised set: {T5, T6, T7}
In minimisation we want to remove all reduncanies, and keep the smallest suite that covers all defined requirements (here, code methods). T6 is necessary because it is the only test that covers M5 and also covers M3. T5 covers M1. And T7 covers M2 and M4. Other suites can create unnecessary coverage overlap.

Prioritised set: [T6, T7, T5, T4, T2, T1, T3]
(T6, T7, T5 should come first, T3 among the last)
In prioritisation we want to ensure covering all different funcitons as soon as possible. Therefore, we should begin with critical cases (either T6, since it uniquely covers M5), then followed by either T5 or T7, as those would cover new methods. We want to end with T3, as it is redundant to T6 that is more valuable.

b. **[7 pts]** Your team needs to prioritize test cases for the regression tests before the next deployment. The time available to run all tests is limited, therefore your team aims to prioritise the test suite. Your colleagues propose two different prioritisation criteria:

- **Alex** suggests prioritizing the **fastest tests first**, to maximize the number of tests executed before the deadline.
- **Taylor** suggests prioritizing the the tests that **covers more different requirements first**, even if some of those tests will take longer to run.

The SUT is a software component for automous driving. For each prioritisation criteria, explain one advantage and one disadvantage in the context of the SUT.

+3 per each advantage / disadvantage pair. +1 for total cohesion of explanation

For Alex's criteria the main advantage is that we can **guarantee a larger number of test executions**, which can be halpful for future decisions. The disavantage is that **we cannot guarantee coverage of different parts of the system**, as the faster tests might all be connected to the same requirement. Therefore, there is higher change of missing different faults, which is not ideal for this SUT (safety-critical)

For Taylor's criteria, the advantage is that we can **execute first those requirements that might be more used by the drivers**, or are more critical to the functionality of the system. The disadvantage is that we **don't have the guarantee that we will execute many tests** (we might even just execute one). Therefore, there is a higher change that the regression test suite won't give meaningful failure history. However, **requirement coverage is preferred** given that this is a safety-critical system

c. **[8 pts]** Your team is preparing to deploy a major update to a popular mobile app used by millions of users daily. The update is in the backend and changing database technologies to support faster queries. You are discussing how to roll out the new version to users, and your team wants to decide between using **A/B Testing** or a **Canary Release**.

Explain which strategy is more suitable for this release. Your answer must contrast between both strategies in the context of this problem.

+1 correct choice, +3 define each concept, +4 justification.

With a **Canary Release**, the update is **gradually rolled out** to a **small subset of users first**. If serious problems are detected, the rollout can be **stopped or rolled back** before affecting all users. In contrast, **A/B Testing** is designed for **comparing different user experiences** (e.g., layout, button color) or measuring behavioral differences between two versions.

The update involves **backend changes** and **switching database technologies**, which can cause **unexpected crashes, data loss, or major performance issues**. Therefore, Canary releases is more suitable.